
MODULE *protocol*

Zcash P2P protocol specification following ZIP – 204.

Models the connection lifecycle between peers: handshake (version/*verack*), keepalive (ping/pong), block synchronization (*inv/getheaders/headers/getdata/block*), and timeout-based disconnection.

Communication uses a message consumption model — each peer has a per-connection *FIFO inbox* and all decisions are based on message payloads only, matching the information constraints of a real network.

The spec verifies one liveness property (all peers eventually reach the same block height) and several safety invariants derived from ZIP – 204 bounds.

EXTENDS *TLC*, *Naturals*, *Sequences*, *FiniteSets*, *messages*

CONSTANT *InitialPeers*

CONSTANT *MaxBlock*

CONSTANT *MaxClock*

CONSTANT *DisconnectTimeout*

CONSTANT *MinPeerProtoVersion*

VARIABLES *nodes*, *clock*

vars $\triangleq$ $\langle nodes, clock \rangle$

See *README* for an explanation of symmetry reduction.

Symmetry $\triangleq$ *Permutations*(*InitialPeers*)

For each initial peer construct a set of all other peers.

OtherPeers $\triangleq$ $[n \in InitialPeers \mapsto InitialPeers \setminus \{n\}]$

Init $\triangleq$

$\wedge clock = 0$

$\wedge \exists blockset \in [InitialPeers \rightarrow (1 \dots MaxBlock)] :$

$nodes = [i \in InitialPeers \mapsto [$

$inbox \mapsto [j \in OtherPeers[i] \mapsto \langle \rangle],$

$conn \mapsto [j \in OtherPeers[i] \mapsto \text{“init”}],$

$ping_nonce \mapsto [j \in OtherPeers[i] \mapsto 0],$

$last_recv_at \mapsto [j \in OtherPeers[i] \mapsto 0],$

$blocks \mapsto 1 \dots blockset[i]$

$]]$

— Handshake —

n sends its version to *m*. The message lands in *m*’s *inbox*.

SendVersion $\triangleq$

$\exists n \in InitialPeers :$

$\exists m \in OtherPeers[n] :$

$\wedge nodes[n].conn[m] = \text{"init"}$
 $\wedge nodes' = [nodes \text{ EXCEPT}$
 $\quad ! [m].inbox[n] = Append(@, MakeVersion(n, m, nodes[n].blocks)),$
 $\quad ! [n].conn[m] = \text{"version_sent"}]$
 $\wedge \text{UNCHANGED } \langle clock \rangle$

n receives m 's version from its *inbox*. Validates the version field:

valid ($\geq MinPeerProtoVersion$) $\rightarrow$ send *verack*, transition to established
 invalid ($< MinPeerProtoVersion$) $\rightarrow$ send reject, reset to *init*

$RecvVersion \triangleq$

$\exists n \in InitialPeers :$

$\exists m \in OtherPeers[n] :$

$\wedge nodes[n].conn[m] = \text{"version_sent"}$
 $\wedge Len(nodes[n].inbox[m]) > 0$
 $\wedge Head(nodes[n].inbox[m]).header.command = \text{"version"}$
 $\wedge LET \ msg \triangleq Head(nodes[n].inbox[m])$

IN

$\vee \wedge msg.payload.version \geq MinPeerProtoVersion$

$\wedge nodes' = [nodes \text{ EXCEPT}$
 $\quad ! [n].inbox[m] = Tail(@),$
 $\quad ! [m].inbox[n] = Append(@, MakeVerack),$
 $\quad ! [n].conn[m] = \text{"established"},$
 $\quad ! [n].last_recv_at[m] = clock]$

$\vee \wedge msg.payload.version < MinPeerProtoVersion$

$\wedge nodes' = [nodes \text{ EXCEPT}$
 $\quad ! [n].inbox[m] = Tail(@),$
 $\quad ! [m].inbox[n] = \langle MakeReject(\text{"version"}) \rangle,$
 $\quad ! [n].conn[m] = \text{"init"},$
 $\quad ! [n].ping_nonce[m] = 0,$
 $\quad ! [n].last_recv_at[m] = clock]$

$\wedge \text{UNCHANGED } \langle clock \rangle$

n receives a *verack* from m . If n is still in *version_sent*, this completes the

handshake. If n already advanced past established (async), just consume the message.

$RecvVerack \triangleq$

$\exists n \in InitialPeers :$

$\exists m \in OtherPeers[n] :$

$\wedge nodes[n].conn[m] \notin \{\text{"init"}\}$
 $\wedge Len(nodes[n].inbox[m]) > 0$
 $\wedge Head(nodes[n].inbox[m]).header.command = \text{"verack"}$
 $\wedge nodes' = [nodes \text{ EXCEPT}$

$\quad ! [n].inbox[m] = Tail(@),$
 $\quad ! [n].conn[m] = \text{IF } @ = \text{"version_sent"} \text{ THEN "established" ELSE } @,$
 $\quad ! [n].last_recv_at[m] = clock]$

$\wedge \text{UNCHANGED } \langle clock \rangle$

n receives an unexpected version from m while already past the handshake.

This models a reconnecting peer whose old connection was torn down unilaterally.

n resets to *init*, matching Zebra's *DuplicateHandshake* disconnect behavior.

$RecvVersionReset \triangleq$

$\exists n \in InitialPeers :$

$\exists m \in OtherPeers[n] :$

$\wedge nodes[n].conn[m] \notin \{ "init", "version_sent" \}$

$\wedge Len(nodes[n].inbox[m]) > 0$

$\wedge Head(nodes[n].inbox[m]).header.command = "version"$

$\wedge nodes' = [nodes \text{ EXCEPT}$

$! [n].inbox[m] = Tail(@),$

$! [n].conn[m] = "init",$

$! [n].ping_nonce[m] = 0,$

$! [n].last_recv_at[m] = clock]$

$\wedge \text{UNCHANGED } \langle clock \rangle$

n discards a non-version message from m while waiting for the handshake.

After a unilateral disconnect, stale messages from the old connection may still be in the *inbox*. Matches Zebra's behavior of ignoring non-version messages during the handshake phase.

$DiscardStaleMessage \triangleq$

$\exists n \in InitialPeers :$

$\exists m \in OtherPeers[n] :$

$\wedge nodes[n].conn[m] = "version_sent"$

$\wedge Len(nodes[n].inbox[m]) > 0$

$\wedge Head(nodes[n].inbox[m]).header.command \notin \{ "version", "verack", "reject" \}$

$\wedge nodes' = [nodes \text{ EXCEPT}$

$! [n].inbox[m] = Tail(@)]$

$\wedge \text{UNCHANGED } \langle clock \rangle$

n receives a reject from m , resetting to *init*.

$RecvReject \triangleq$

$\exists n \in InitialPeers :$

$\exists m \in OtherPeers[n] :$

$\wedge Len(nodes[n].inbox[m]) > 0$

$\wedge Head(nodes[n].inbox[m]).header.command = "reject"$

$\wedge nodes' = [nodes \text{ EXCEPT}$

$! [n].inbox[m] = Tail(@),$

$! [n].conn[m] = "init",$

$! [n].ping_nonce[m] = 0,$

$! [n].last_recv_at[m] = clock]$

$\wedge \text{UNCHANGED } \langle clock \rangle$

— *Keepalive* —

n sends a ping to m when idle.

$SendPing \triangleq$
 $\exists n \in InitialPeers :$
 $\exists m \in OtherPeers[n] :$
 $\wedge nodes[n].conn[m] \notin \{ "init", "version_sent" \}$
 $\wedge nodes[n].last_recv_at[m] \leq clock - 3$
 $\wedge nodes[n].ping_nonce[m] = 0$
 $\wedge nodes' = [nodes \text{ EXCEPT}$
 $\quad ! [m].inbox[n] = Append(@, MakePing),$
 $\quad ! [n].ping_nonce[m] = clock]$
 $\wedge UNCHANGED \langle clock \rangle$

n receives a ping from m and immediately replies with pong.

$RecvPing \triangleq$
 $\exists n \in InitialPeers :$
 $\exists m \in OtherPeers[n] :$
 $\wedge nodes[n].conn[m] \notin \{ "init", "version_sent" \}$
 $\wedge Len(nodes[n].inbox[m]) > 0$
 $\wedge Head(nodes[n].inbox[m]).header.command = "ping"$
 $\wedge nodes' = [nodes \text{ EXCEPT}$
 $\quad ! [n].inbox[m] = Tail(@),$
 $\quad ! [m].inbox[n] = Append(@, MakePong),$
 $\quad ! [n].last_recv_at[m] = clock]$
 $\wedge UNCHANGED \langle clock \rangle$

n receives a pong from m , completing the keepalive round-trip.

$RecvPong \triangleq$
 $\exists n \in InitialPeers :$
 $\exists m \in OtherPeers[n] :$
 $\wedge nodes[n].ping_nonce[m] \neq 0$
 $\wedge Len(nodes[n].inbox[m]) > 0$
 $\wedge Head(nodes[n].inbox[m]).header.command = "pong"$
 $\wedge nodes' = [nodes \text{ EXCEPT}$
 $\quad ! [n].inbox[m] = Tail(@),$
 $\quad ! [n].ping_nonce[m] = 0,$
 $\quad ! [n].last_recv_at[m] = clock]$
 $\wedge UNCHANGED \langle clock \rangle$

— Block sync —

n announces its inventory to m after handshake.

$SendInv \triangleq$
 $\exists n \in InitialPeers :$
 $\exists m \in OtherPeers[n] :$
 $\wedge nodes[n].conn[m] = "established"$
 $\wedge nodes[n].blocks \neq \{ \}$
 $\wedge nodes' = [nodes \text{ EXCEPT}$

$$\begin{aligned}
& ![m].inbox[n] &= Append(@, MakeInv(nodes[n].blocks)), \\
& ![n].conn[m] &= "inv_sent", \\
& ![n].last_recv_at[m] &= clock \\
& \wedge \text{UNCHANGED } \langle clock \rangle
\end{aligned}$$

n receives an *inv* from m . Both peers must have sent their invs first (n is *inv_sent*).

Inspects the *payload* to decide: sync or already caught up.

$RecvInv \triangleq$

$$\begin{aligned}
& \exists n \in InitialPeers : \\
& \quad \exists m \in OtherPeers[n] : \\
& \quad \quad \wedge nodes[n].conn[m] = "inv_sent" \\
& \quad \quad \wedge Len(nodes[n].inbox[m]) > 0 \\
& \quad \quad \wedge Head(nodes[n].inbox[m]).header.command = "inv" \\
& \quad \quad \wedge LET msg \triangleq Head(nodes[n].inbox[m]) \\
& \quad \quad \quad advertised \triangleq msg.payload.inventory[1].hash \\
& \quad \quad IN \\
& \quad \quad \quad \vee \wedge advertised > Cardinality(nodes[n].blocks) \\
& \quad \quad \quad \quad \wedge nodes' = [nodes \text{ EXCEPT} \\
& \quad \quad \quad \quad \quad ![n].inbox[m] = Tail(@), \\
& \quad \quad \quad \quad \quad ![m].inbox[n] = Append(@, MakeGetHeaders(nodes[n].blocks)), \\
& \quad \quad \quad \quad \quad ![n].conn[m] = "getheaders_sent", \\
& \quad \quad \quad \quad \quad ![n].last_recv_at[m] = clock \\
& \quad \quad \quad \vee \wedge advertised \leq Cardinality(nodes[n].blocks) \\
& \quad \quad \quad \quad \wedge nodes' = [nodes \text{ EXCEPT} \\
& \quad \quad \quad \quad \quad ![n].inbox[m] = Tail(@), \\
& \quad \quad \quad \quad \quad ![n].conn[m] = "synced"] \\
& \quad \quad \wedge \text{UNCHANGED } \langle clock \rangle
\end{aligned}$$

n receives a *getheaders* request from m and responds with its own headers.

$RecvGetHeaders \triangleq$

$$\begin{aligned}
& \exists n \in InitialPeers : \\
& \quad \exists m \in OtherPeers[n] : \\
& \quad \quad \wedge nodes[n].conn[m] \notin \{ "init", "version_sent" \} \\
& \quad \quad \wedge Len(nodes[n].inbox[m]) > 0 \\
& \quad \quad \wedge Head(nodes[n].inbox[m]).header.command = "getheaders" \\
& \quad \quad \wedge nodes' = [nodes \text{ EXCEPT} \\
& \quad \quad \quad ![n].inbox[m] = Tail(@), \\
& \quad \quad \quad ![m].inbox[n] = Append(@, MakeHeaders(nodes[n].blocks)), \\
& \quad \quad \quad ![n].last_recv_at[m] = clock \\
& \quad \quad \wedge \text{UNCHANGED } \langle clock \rangle
\end{aligned}$$

n receives headers from m (response to its *getheaders*), sends *getdata*.

$RecvHeaders \triangleq$

$$\begin{aligned}
& \exists n \in InitialPeers : \\
& \quad \exists m \in OtherPeers[n] : \\
& \quad \quad \wedge nodes[n].conn[m] = "getheaders_sent"
\end{aligned}$$

$$\begin{aligned}
& \wedge \text{Len}(\text{nodes}[n].\text{inbox}[m]) > 0 \\
& \wedge \text{Head}(\text{nodes}[n].\text{inbox}[m]).\text{header.command} = \text{"headers"} \\
& \wedge \text{nodes}' = [\text{nodes} \text{ EXCEPT} \\
& \quad ! [n].\text{inbox}[m] = \text{Tail}(@), \\
& \quad ! [m].\text{inbox}[n] = \text{Append}(@, \text{MakeGetData}(\text{nodes}[n].\text{blocks})), \\
& \quad ! [n].\text{conn}[m] = \text{"getdata_sent"}, \\
& \quad ! [n].\text{last_recv_at}[m] = \text{clock}] \\
& \wedge \text{UNCHANGED } \langle \text{clock} \rangle
\end{aligned}$$

n receives a *getdata* request from *m* and responds with a block.

RecvGetData $\triangleq$

$$\begin{aligned}
& \exists n \in \text{InitialPeers} : \\
& \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \wedge \text{nodes}[n].\text{conn}[m] \notin \{\text{"init"}, \text{"version_sent"}\} \\
& \quad \wedge \text{Len}(\text{nodes}[n].\text{inbox}[m]) > 0 \\
& \quad \wedge \text{Head}(\text{nodes}[n].\text{inbox}[m]).\text{header.command} = \text{"getdata"} \\
& \quad \wedge \text{nodes}' = [\text{nodes} \text{ EXCEPT} \\
& \quad \quad ! [n].\text{inbox}[m] = \text{Tail}(@), \\
& \quad \quad ! [m].\text{inbox}[n] = \text{Append}(@, \text{MakeBlock}(\text{nodes}[n].\text{blocks})), \\
& \quad \quad ! [n].\text{last_recv_at}[m] = \text{clock}] \\
& \quad \wedge \text{UNCHANGED } \langle \text{clock} \rangle
\end{aligned}$$

n receives a block from *m*, adds it to its local chain.

RecvBlock $\triangleq$

$$\begin{aligned}
& \exists n \in \text{InitialPeers} : \\
& \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \wedge \text{nodes}[n].\text{conn}[m] = \text{"getdata_sent"} \\
& \quad \wedge \text{Len}(\text{nodes}[n].\text{inbox}[m]) > 0 \\
& \quad \wedge \text{Head}(\text{nodes}[n].\text{inbox}[m]).\text{header.command} = \text{"block"} \\
& \quad \wedge \text{LET } \text{newBlocks} \triangleq \text{nodes}[n].\text{blocks} \cup \{\text{Cardinality}(\text{nodes}[n].\text{blocks}) + 1\} \\
& \quad \quad \text{msg} \triangleq \text{Head}(\text{nodes}[n].\text{inbox}[m]) \\
& \quad \text{IN } \text{nodes}' = [\text{nodes} \text{ EXCEPT} \\
& \quad \quad ! [n].\text{inbox}[m] = \text{Tail}(@), \\
& \quad \quad ! [n].\text{last_recv_at}[m] = \text{clock}, \\
& \quad \quad ! [n].\text{blocks} = \text{newBlocks}, \\
& \quad \quad ! [n].\text{conn}[m] = \text{IF } \text{Cardinality}(\text{newBlocks}) < \text{msg.payload.prev_block} \\
& \quad \quad \quad \text{THEN "block_received"} \\
& \quad \quad \quad \text{ELSE "synced"}] \\
& \quad \wedge \text{UNCHANGED } \langle \text{clock} \rangle
\end{aligned}$$

n has received a block but still needs more — sends another *getdata*.

SendGetData $\triangleq$

$$\begin{aligned}
& \exists n \in \text{InitialPeers} : \\
& \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \wedge \text{nodes}[n].\text{conn}[m] = \text{"block_received"} \\
& \quad \wedge \text{nodes}' = [\text{nodes} \text{ EXCEPT}
\end{aligned}$$

$$\begin{aligned}
& ![m].inbox[n] &= Append(@, MakeGetData(nodes[n].blocks)), \\
& ![n].conn[m] &= "getdata_sent", \\
& ![n].last_recv_at[m] &= clock \\
& \wedge \text{UNCHANGED } \langle clock \rangle
\end{aligned}$$

— Disconnection —

Unilateral disconnect: models *TCP* FIN — only the detecting side resets.

The remote peer's *inbox* from *n* is also cleared (the *TCP* pipe is broken, so messages in transit are lost), but *m*'s connection state is unchanged — *m* will independently detect the timeout and disconnect on its own.

This matches Zebra's implementation where each side detects idleness independently.

Disconnect $\triangleq$

$$\begin{aligned}
& \exists n \in \text{InitialPeers} : \\
& \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \wedge nodes[n].conn[m] \notin \{ "init" \} \\
& \quad \wedge clock - nodes[n].last_recv_at[m] > \text{DisconnectTimeout} \\
& \quad \wedge nodes' = [nodes \text{ EXCEPT} \\
& \quad \quad ![n].inbox[m] &= \langle \rangle, \\
& \quad \quad ![m].inbox[n] &= \langle \rangle, \\
& \quad \quad ![n].conn[m] &= "init", \\
& \quad \quad ![n].ping_nonce[m] &= 0, \\
& \quad \quad ![n].last_recv_at[m] &= clock \\
& \quad \wedge \text{UNCHANGED } \langle clock \rangle
\end{aligned}$$

Tick $\triangleq$

$$\begin{aligned}
& \wedge clock < \text{MaxClock} \\
& \wedge clock' = clock + 1 \\
& \wedge \text{UNCHANGED } \langle nodes \rangle
\end{aligned}$$

Next $\triangleq$

$$\begin{aligned}
& \vee \text{Tick} \\
& \vee \text{SendVersion} \\
& \vee \text{RecvVersion} \\
& \vee \text{RecvVersionReset} \\
& \vee \text{DiscardStaleMessage} \\
& \vee \text{RecvVerack} \\
& \vee \text{RecvReject} \\
& \vee \text{SendPing} \\
& \vee \text{RecvPing} \\
& \vee \text{RecvPong} \\
& \vee \text{SendInv} \\
& \vee \text{RecvInv} \\
& \vee \text{RecvGetHeaders}
\end{aligned}$$

$\vee \text{RecvHeaders}$
 $\vee \text{RecvGetData}$
 $\vee \text{RecvBlock}$
 $\vee \text{SendGetData}$
 $\vee \text{Disconnect}$

$\text{Spec} \triangleq$
 Init
 $\wedge \square[\text{Next}]_{\text{vars}}$
 $\wedge \text{WF}_{\text{vars}}(\text{Next})$

Liveness property: all peers eventually reach the same block height.

$\text{EventualConsensus} \triangleq \Diamond \forall i, j \in \text{InitialPeers} : \text{nodes}[i].\text{blocks} = \text{nodes}[j].\text{blocks}$

ZIP – 0204 invariants: safety properties checked at every reachable state.

inv and getdata inventory vectors must not exceed 50,000 entries (ZIP – 0204 §4).

$\text{InvCountBounded} \triangleq$
 $\forall n \in \text{InitialPeers} :$
 $\forall m \in \text{OtherPeers}[n] :$
 $\forall i \in 1 \dots \text{Len}(\text{nodes}[n].\text{inbox}[m]) :$
 $\text{nodes}[n].\text{inbox}[m][i].\text{header}.command = \text{"inv"}$
 $\Rightarrow \text{nodes}[n].\text{inbox}[m][i].\text{payload}.count \leq 50000$

$\text{GetDataCountBounded} \triangleq$
 $\forall n \in \text{InitialPeers} :$
 $\forall m \in \text{OtherPeers}[n] :$
 $\forall i \in 1 \dots \text{Len}(\text{nodes}[n].\text{inbox}[m]) :$
 $\text{nodes}[n].\text{inbox}[m][i].\text{header}.command = \text{"getdata"}$
 $\Rightarrow \text{nodes}[n].\text{inbox}[m][i].\text{payload}.count \leq 50000$

headers messages must not carry more than 160 block headers (ZIP – 0204 §4).

$\text{HeadersCountBounded} \triangleq$
 $\forall n \in \text{InitialPeers} :$
 $\forall m \in \text{OtherPeers}[n] :$
 $\forall i \in 1 \dots \text{Len}(\text{nodes}[n].\text{inbox}[m]) :$
 $\text{nodes}[n].\text{inbox}[m][i].\text{header}.command = \text{"headers"}$
 $\Rightarrow \text{nodes}[n].\text{inbox}[m][i].\text{payload}.count \leq 160$

Peers must speak at least $\text{MinPeerProtoVersion}$ (ZIP – 0204 §3).

$\text{VersionBounded} \triangleq$
 $\forall n \in \text{InitialPeers} :$
 $\forall m \in \text{OtherPeers}[n] :$
 $\forall i \in 1 \dots \text{Len}(\text{nodes}[n].\text{inbox}[m]) :$
 $\text{nodes}[n].\text{inbox}[m][i].\text{header}.command = \text{"version"}$
 $\Rightarrow \text{nodes}[n].\text{inbox}[m][i].\text{payload}.version \geq \text{MinPeerProtoVersion}$

A non-zero ping nonce implies the connection is past the handshake.

$PingOnEstablished \triangleq$

$\forall n \in InitialPeers :$

$\forall m \in OtherPeers[n] :$

$nodes[n].ping_nonce[m] \neq 0$

$\Rightarrow nodes[n].conn[m] \notin \{ "init", "version_sent" \}$

A peer only enters sync states when it actually has fewer blocks than its partner.

NOTE: uses $\leq$ (not $<$) because with message consumption, m could gain blocks from a third peer between when n enters a sync state and when we check.

$SyncDirection \triangleq$

$\forall n \in InitialPeers :$

$\forall m \in OtherPeers[n] :$

$nodes[n].conn[m] \in \{ "getheaders_sent", "getdata_sent", "block_received" \}$

$\Rightarrow Cardinality(nodes[n].blocks) \leq Cardinality(nodes[m].blocks)$